

Lab 2: C Programming Basics

Lab Report and Knowledge Check

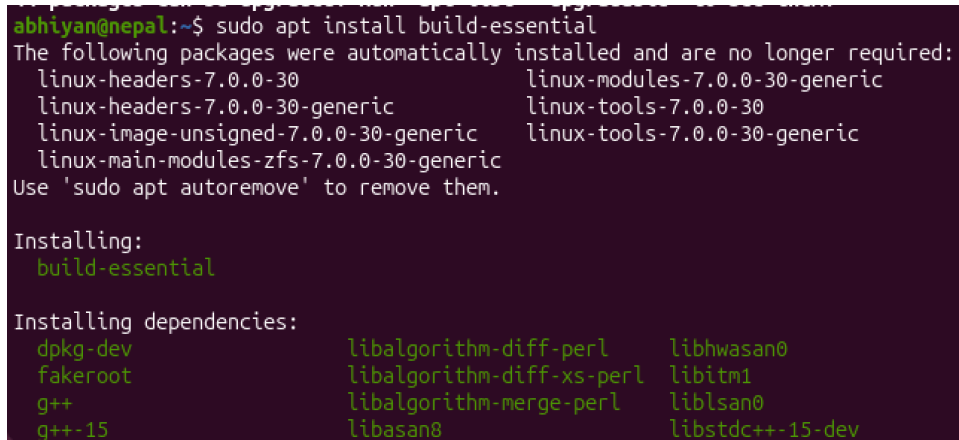
I. Introduction

This report documents the practical work completed for Lecture 2 of ST5003CMD Operating Systems, Security and Networks. The lab focused on writing a basic C program, installing and using the GCC toolchain, walking through the four stages of C compilation individually, and examining how a program's return value is reported to the shell as its exit status.

II. Environment Setup

Before writing any C code, the build-essential package was installed to provide GCC and the associated development tools, and the installed compiler version was verified.

A. Installing build-essential



```
abhiyan@nepal:~$ sudo apt install build-essential
The following packages were automatically installed and are no longer required:
  linux-headers-7.0.0-30          linux-modules-7.0.0-30-generic
  linux-headers-7.0.0-30-generic  linux-tools-7.0.0-30
  linux-image-unsigned-7.0.0-30-generic  linux-tools-7.0.0-30-generic
  linux-main-modules-zfs-7.0.0-30-generic
Use 'sudo apt autoremove' to remove them.

Installing:
  build-essential

Installing dependencies:
  dpkg-dev          libalgorithm-diff-perl  libhwasan0
  fakeroot          libalgorithm-diff-xs-perl  libitm1
  g++               libalgorithm-merge-perl  liblsan0
  g++-15            libasan8                 libstdc++-15-dev
```

Figure 1. Installing the build-essential package with apt.

B. Verifying the GCC Installation

```
No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
abhiyan@nepal:~$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

abhiyan@nepal:~$
```

Figure 2. Checking the installed GCC version with gcc --version.

Summary: build-essential was installed successfully, and gcc --version confirmed GCC 15.2.0 was available on the system.

III. Writing the Hello World Program

A source file, hello.c, was created using the nano text editor. The program includes the stdio.h header, prints a greeting using printf, and returns 0 from main() to signal successful execution.

```
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

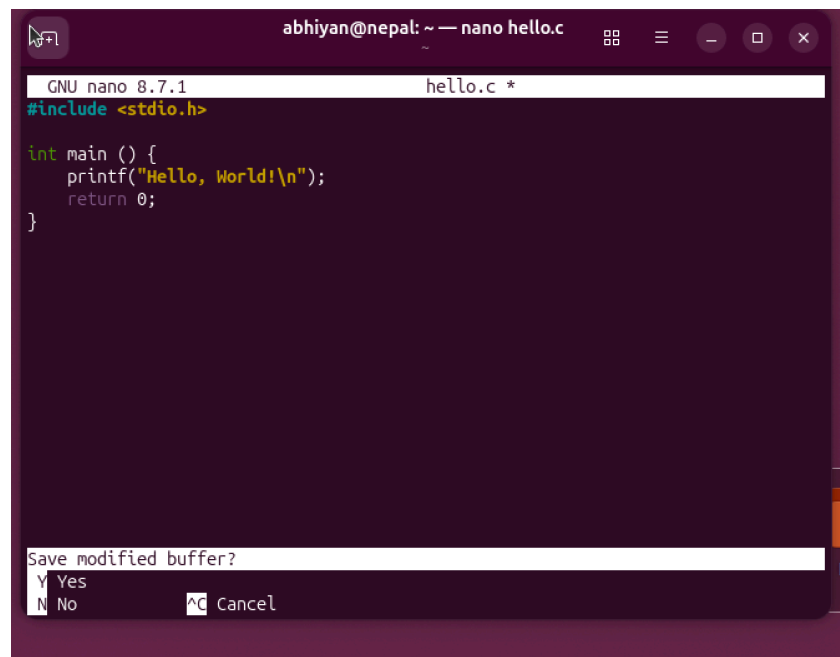


Figure 3. hello.c opened in nano, showing the basic program structure.

Summary: A new C source file was created and the basic Hello World program was written and saved.

IV. The Four Stages of Compilation

GCC turns a C source file into an executable in four separate stages: preprocessing, compilation to assembly, assembly to an object file, and linking. Each stage was run manually using the `-E`, `-S`, and `-c` flags to observe the intermediate output files.

A. Stage 1 — Preprocessing (.i)

The preprocessor expands `#include` directives and macros, producing a `.i` file.

```
gcc -E hello.c -o hello.i
```

A terminal window with a dark purple background. It shows the command `gcc -E hello.c -o hello.i` being executed. The output is "compilation terminated." followed by the prompt `abhiyan@nepal:~$`. The user then runs `nano hello.c`, which also shows "compilation terminated." and the prompt. Finally, the user runs `gcc -E hello.c -o hello.i`, which produces the same output.

Figure 4. Running the preprocessing stage to generate `hello.i`.

B. Stage 2 — Compilation to Assembly (.s)

The preprocessed file is translated into assembly code, producing a `.s` file.

```
gcc -S hello.i -o hello.s
```

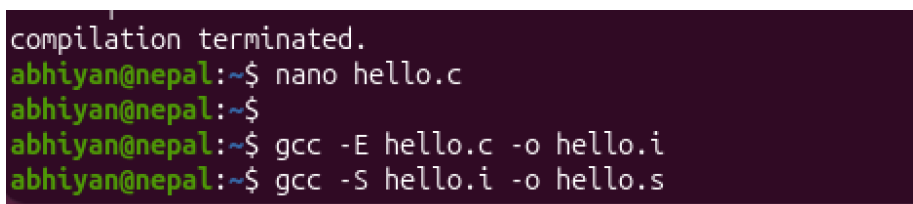
A terminal window with a dark purple background. It shows the command `gcc -S hello.i -o hello.s` being executed. The output is "compilation terminated." followed by the prompt `abhiyan@nepal:~$`. The user then runs `nano hello.c`, which also shows "compilation terminated." and the prompt. Finally, the user runs `gcc -E hello.c -o hello.i` and `gcc -S hello.i -o hello.s`, both of which produce the same output.

Figure 5. Compiling the preprocessed file into assembly (`hello.s`).

C. A Common Mistake — Linking Before Assembling

Attempting to link directly from `hello.o` at this point failed, because the assembly stage (producing the actual `hello.o` object file) had not been run yet — only `hello.i` and `hello.s` existed so far. The linker reported that `hello.o` could not be found.

```
gcc hello.o -o hello
# /usr/bin/aarch64-linux-gnu-ld.bfd: cannot find hello.o: No such file or directory
# collect2: error: ld returned 1 exit status
```

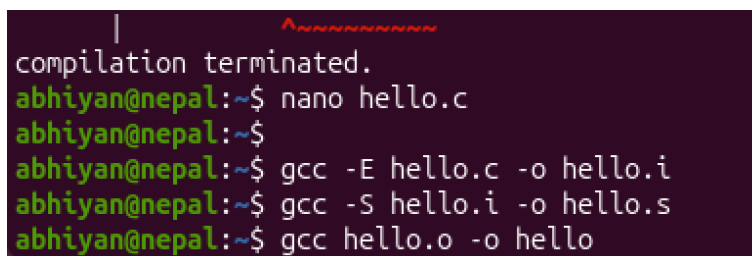
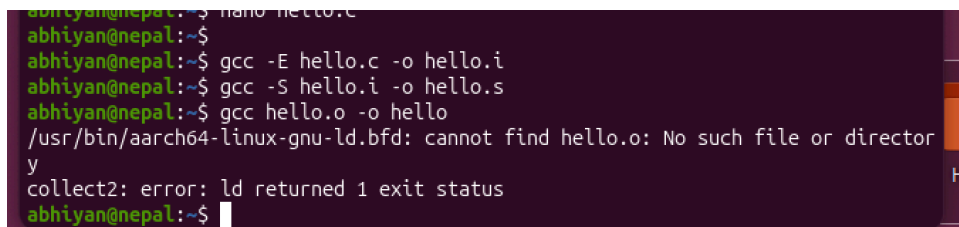
A terminal window with a dark purple background. It shows the command `gcc hello.o -o hello` being executed. The output is "compilation terminated." followed by the prompt `abhiyan@nepal:~$`. The user then runs `nano hello.c`, which also shows "compilation terminated." and the prompt. Finally, the user runs `gcc -E hello.c -o hello.i`, `gcc -S hello.i -o hello.s`, and `gcc hello.o -o hello`, all of which produce the same output.

Figure 6. Linking error caused by skipping the assembly stage.

D. Stage 3 & 4 — Assembling and Linking

Running the assembler produced the missing `hello.o` object file, after which linking succeeded and the program executed correctly.

```
gcc -c hello.s -o hello.o
gcc hello.o -o hello
./hello
```



```
abhiyan@nepal:~$ nano hello.c
abhiyan@nepal:~$ gcc -E hello.c -o hello.i
abhiyan@nepal:~$ gcc -S hello.i -o hello.s
abhiyan@nepal:~$ gcc hello.o -o hello
/usr/bin/aarch64-linux-gnu-ld.bfd: cannot find hello.o: No such file or directory
collect2: error: ld returned 1 exit status
abhiyan@nepal:~$
```

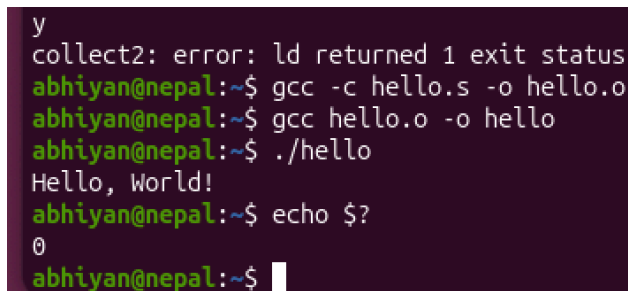
Figure 7. Assembling `hello.s` into `hello.o`, linking, and running the executable, producing "Hello, World!".

Summary: All four compilation stages — preprocessing, compiling, assembling, and linking — were executed as separate GCC commands, and the resulting executable ran correctly once every intermediate file existed.

V. Return Values and Exit Status

The integer returned by `main()` is passed back to the shell as the program's exit status, which can be inspected using `echo $?`. A return value of 0 conventionally signals success.

```
./hello
echo $?
```



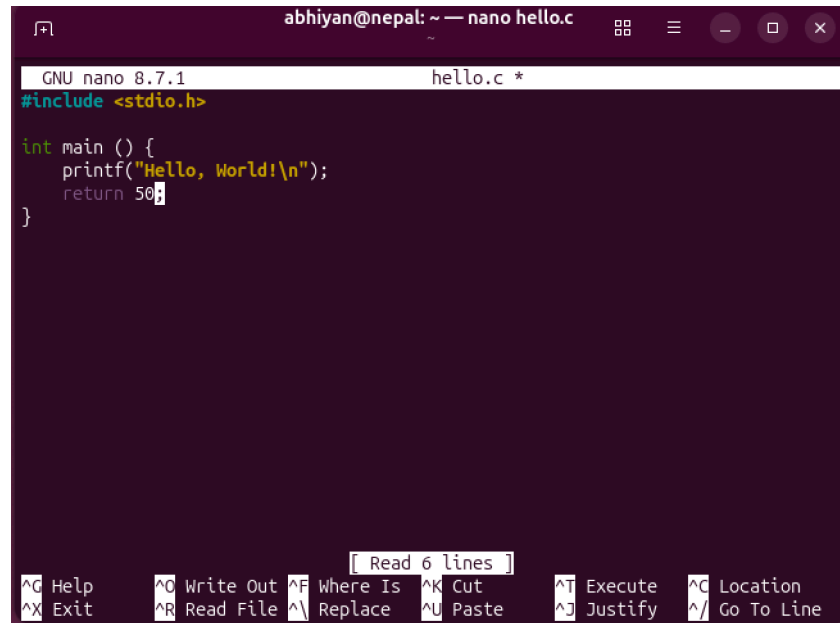
```
y
collect2: error: ld returned 1 exit status
abhiyan@nepal:~$ gcc -c hello.s -o hello.o
abhiyan@nepal:~$ gcc hello.o -o hello
abhiyan@nepal:~$ ./hello
Hello, World!
abhiyan@nepal:~$ echo $?
0
abhiyan@nepal:~$
```

Figure 8. Running `hello` and confirming an exit status of 0 with `echo $?`.

Summary: After compiling and running `hello.c` with return 0, `echo $?` confirmed an exit status of 0, indicating successful execution.

VI. Testing a Non-Zero Return Value

To observe how a non-zero return value is reported, `hello.c` was edited in `nano` so that `main()` returns 50 instead of 0, then recompiled and re-run.



```
abhiyan@nepal: ~ — nano hello.c
GNU nano 8.7.1 hello.c *
#include <stdio.h>

int main () {
    printf("Hello, World!\n");
    return 50;
}

[ Read 6 lines ]
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

Figure 9. Editing hello.c in nano to change the return value to 50.

```
gcc hello.c -o hello
./hello
echo $?
```

```
compilation terminated.
abhiyan@nepal:~$ nano hello.c
abhiyan@nepal:~$
abhiyan@nepal:~$ gcc -E hello.c -o hello.i
abhiyan@nepal:~$ gcc -S hello.i -o hello.s
abhiyan@nepal:~$ gcc hello.o -o hello
/usr/bin/aarch64-linux-gnu-ld.bfd: cannot find hello.o: No such file or director
y
collect2: error: ld returned 1 exit status
abhiyan@nepal:~$ gcc -c hello.s -o hello.o
abhiyan@nepal:~$ gcc hello.o -o hello
abhiyan@nepal:~$ ./hello
Hello, World!
abhiyan@nepal:~$ echo $?
0
abhiyan@nepal:~$ nano hello.c
abhiyan@nepal:~$ gcc hello.c -o hello
abhiyan@nepal:~$ ./hello
Hello, World!
abhiyan@nepal:~$ echo $?
50
abhiyan@nepal:~$
```

Figure 10. Recompiling and rerunning hello, with echo \$? now reporting 50.

Summary: After changing the return statement to return 50, recompiling, and rerunning the program, echo \$? reported an exit status of 50, confirming that the value returned by main() is passed directly to the shell.

VII. Knowledge Check Questions

Q: What is the purpose of the #include directive in C programming? Provide an example of its usage.

A: #include tells the preprocessor to insert the contents of another file — typically a header — into the source file before compilation. It is how a program gains access to declarations for library functions it uses. For example, #include <stdio.h> pulls in the declarations needed for printf() and scanf(), which is why it appears at the top of hello.c.

Q: Explain the difference between the four stages of compilation. What file extension is produced at each stage?

A: Preprocessing (gcc -E) expands macros and #include directives, producing a .i file. Compilation (gcc -S) translates the preprocessed C code into assembly language, producing a .s file. Assembly (gcc -c) converts the assembly code into machine code stored in an object file with a .o extension. Linking (gcc file.o -o program) combines the object file with any required libraries and startup code to produce the final executable, which has no extension on Linux (e.g. hello).

Q: What does a return value of 0 from the main() function indicate to the operating system? What about a non-zero return value?

A: A return value of 0 tells the operating system that the program finished successfully with no errors. A non-zero return value signals that the program terminated abnormally or encountered a specific error condition, and the particular number can be used by scripts or the calling process to identify what went wrong. This was demonstrated directly: hello returned 0 after normal execution, and echo \$? reported 50 once main() was changed to return 50.

Q: Write a C program that declares an integer variable, asks the user to input a value, and displays the entered value back to the user.

A: The following program satisfies these requirements:

```
#include <stdio.h>

int main() {
    int value;
    printf("Enter an integer: ");
    scanf("%d", &value);
    printf("You entered: %d\n", value);
    return 0;
}
```

Q: What GCC command would you use to (1) only preprocess a file without compiling, (2) generate assembly code but not assemble it, and (3) create an object file without linking?

A: (1) gcc -E file.c -o file.i stops after preprocessing. (2) gcc -S file.c -o file.s stops after generating assembly code. (3) gcc -c file.c -o file.o stops after assembling, producing an object file without linking it into an executable.

Q: Explain why C is called a "compiled language" and how this differs from interpreted languages.

A: C is called a compiled language because its source code is translated in advance, by GCC in this lab, through the preprocessing, compilation, assembly, and linking stages into a standalone machine-code executable that the CPU can run directly. An interpreted language, by contrast, has no separate build step: an interpreter reads and executes the source code (or a lightweight bytecode form of it) one statement at a time at runtime, which is generally slower but skips the compile-then-run workflow seen throughout this lab.

VIII. Lab Tasks Completed

1. Installed build-essential and verified the GCC version (Figures 1–2).
2. Wrote and saved a basic Hello World C program in nano (Figure 3).
3. Ran each of the four compilation stages individually with `gcc -E`, `-S`, `-c`, and the final link step, and diagnosed a linker error caused by skipping the assembly stage (Figures 4–7).
4. Compiled and ran `hello.c` with `return 0` and verified the exit status using `echo $?` (Figure 8).
5. Modified the program to `return 50`, recompiled it, and confirmed the new exit status was reported correctly (Figures 9–10).

IX. Conclusion

This lab provided hands-on experience with the fundamentals of C program structure, the GCC toolchain, and the multi-stage compilation process. Running preprocessing, compilation, assembly, and linking as separate commands made clear how each stage depends on the output of the one before it — a dependency that surfaced directly when linking was attempted before the object file existed. Testing both a zero and a non-zero return value also showed concretely how a C program communicates its outcome to the operating system through its exit status.

GitHub link: https://github.com/abhiyan0/batch38_ai_OS_abhiyansharma.git